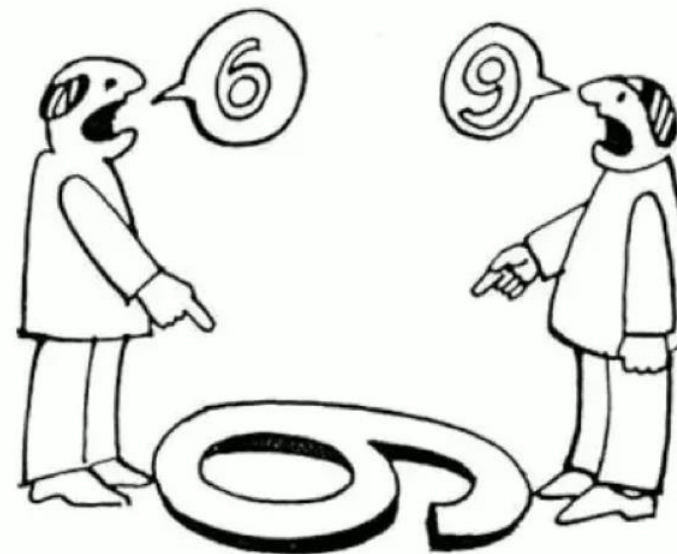


Лекция 3

Критерии качества требований

Требования

- ☒ Атомарность
- ☒ Необходимость
- ☒ Прослеживаемость
- ☒ Модифицируемость
- ☒ Понятность



☐ Однозначность

Как оценить качество требований?

Требования считаются качественными, если они **отражают потребности заинтересованных сторон и предоставляют достаточно информации для создания программного обеспечения**, которое удовлетворяет целям продукта или проекта.

На любом проекте **некачественные требования приводят к нежелательным последствиям**, таким как:

- Множество раундов разъяснений и уточнений;
- Сложности в планировании;
- Частые переделки;
- Превышение времени и бюджета;
- Сложности в поддержке требований;
- Несчастливая команда проекта;
- Недовольный клиент.



Дать определение качеству требований и измерить это качество, достаточно трудно.

Важно понимать, что критерии качества требований - это полезный инструмент, который можно и нужно использовать для улучшения эффективности работы.

Развлекательное видео, которое просто и наглядно демонстрирует почему качественные требования так важны



[Exact Instructions Challenge - THIS is why my kids hate me. | Josh Darnit – YouTube](#)

Какие существуют критерии качества?

Существует масса списков критериев качества требований, составленных различными авторами, и "заточенных" под разные методологии разработки. Вот лишь некоторые из них:

По BABOK:

Atomic (Атомарное)
Complete (Полное)
Consistent (Непротиворечивое)
Concise (Краткое)
Feasible (Выполнимое)
Unambiguous (Однозначное)
Testable (Тестируемое)
Prioritized (Приоритизированное)
Understandable (Понятное)

BABOK – руководство по своду знаний по бизнес-анализу

По Вигерсу:

Correct (Верное)
Feasible (Выполнимое)
Necessary (Необходимое)
Prioritized (Приоритизированное)
Unambiguous (Однозначное)
Verifiable (Тестируемое)

INVEST (Bill Wake):

Independent (Независимое)
Negotiable (Обсуждаемое)
Valuable (Полезное)
Estimable (Поддающееся оценке)
Small (Компактное)
Testable (Тестируемое)

Requirements Management Using IBM Rational RequisitePro*:

Unambiguous (Однозначное)
Testable (verifiable) (Тестируемое)
Clear (Четкое)
Correct (Верное)
Understandable (Понятное)
Feasible (Выполнимое)
Independent (Независимое)
Atomic (Атомарное)
Necessary (Необходимое)
Implementation-free (abstract) (Абстрактное)
Consistent (Согласующееся)
Nonredundant (Не избыточное)
Complete (Полное)

* Инструмент IBM Rational RequisitePro предназначен для организации работы аналитиков и автоматизации их деятельности в области управления требованиями.

Не смотря на различия формулировок можно проследить сходство.

<https://habr.com/ru/articles/842296/>

Международный институт
бизнес-анализа «BABOK»

1. Полнота
2. Непротиворечивость
3. Корректность
4. Недвусмысленность
5. Выполнимость
6. Проверяемость
7. Приоретизированность

Атомарность

Понятность

С. Куликов «Тестирование
программного обеспечения.
Базовый курс»

1. Полнота
2. Непротиворечивость
3. Корректность
4. Недвусмысленность
5. Выполнимость
6. Проверяемость
7. Приоретизированность

Атомарность

Необходимость

Прослеживаемость

Модифицируемость

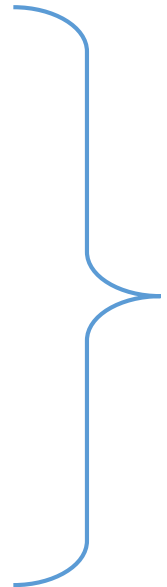
Карл Вигерс и Джой Битти
«Разработка требований к
программному обеспечению»

1. Полнота
2. Непротиворечивость
3. Корректность
4. Недвусмысленность
5. Выполнимость
6. Проверяемость
7. Приоретизированность

Необходимость

Прослеживаемость

Модифицируемость



1. ПОЛНОТА (или ЗАВЕРШЕННОСТЬ).

Каждое требование должно содержать всю информацию, необходимую для его понимания, не оставлять пробелов или недомолвок. Этот критерий относится и к бизнес- и к техническим требованиям.

Пример 1: *Пользователь может загрузить данные в формате PDF, CSV и т.д.*

Неясно, какие ещё форматы поддерживаются помимо указанных. Что подразумевается под «и т.д.»?

Полное требование: "Пользователь может загрузить данные в форматах PDF, CSV и JSON."

Пример 2: *Приложение должно работать на всех популярных мобильных платформах.*

Неясно, какие платформы считаются «популярными».

Полное требование: "Приложение должно поддерживаться на платформах Android версии 10.0 и выше, iOS версии 13.0 и выше."

Пример 3: *Система должна обрабатывать запросы с высокой производительностью.*

Не указано, что конкретно подразумевается под «высокой производительностью» (какие показатели важны).

Полное требование: "Система должна обрабатывать до 1000 запросов в секунду с задержкой не более 200 миллисекунд."

2. НЕПРОТИВОРЕЧИВОСТЬ (или СОГЛАСОВАННОСТЬ).

Требования не должны противоречить друг другу. Обнаружение несоответствий может быть крайне затруднительным, если требования к одной и той же функциональности продукта находятся в разных местах.

Пример 1:

- *Пользователь должен быть автоматически разлогинен после 15 минут бездействия.*
- *Пользователь должен оставаться в системе до тех пор, пока не завершит редактирование документа.*

Противоречие: Первое требование подразумевает автоматический выход по истечении времени, а другое не допускает выхода во время работы над документом.

Непротиворечивые требования: Пользователь должен быть выведен из системы через 15 минут бездействия, за исключением времени активного редактирования документа.

Пример 2:

- *Отчет должен генерироваться и отправляться пользователю мгновенно.*
- *Генерация отчета может занимать до 30 минут в зависимости от объема данных.*

Непротиворечивые требования: Отчет должен генерироваться мгновенно при объеме данных до 200 килобайт и отправляться в течение 30 минут при большем объеме.

3. КОРРЕКТНОСТЬ

Требования должны полностью удовлетворять нужды заинтересованных сторон и отвечать их запросам.

Пример 1: *Пользователь должен быть в состоянии настроить двухфакторную аутентификацию для повышения безопасности*

Это требование к пользователю, а не к программе. Мы делаем продукт и можем диктовать ему требования, но не можем контролировать состояние его пользователей.

Корректное требование: Приложение должно поддерживать двухфакторную аутентификацию и предоставлять пользователю возможность включить эту функцию в настройках безопасности.

Пример 2: *После завершения загрузки файла пользователь должен увидеть пагинацию с сообщением о том, что загрузка прошла успешно.*

Пагинация относится к разбиению контента на страницы, что не имеет смысла в контексте завершения загрузки файла. Вместо этого должно быть указано, что пользователь должен увидеть сообщение о том, что загрузка прошла успешно.

Корректное требование: После завершения загрузки файла пользователь должен увидеть уведомление о том, что загрузка прошла успешно.

4. НЕДВУСМЫСЛЕННОСТЬ (или ОДНОЗНАЧНОСТЬ)

Разговорный язык может содержать двусмысленности, которые обычно проявляются в двух формах: когда одно требование можно интерпретировать по-разному и когда разные люди понимают одно и то же требование по-своему. Важно, чтобы требования были сформулированы однозначно, без жаргона, аббревиатур и неясных фраз, чтобы не возникало различных трактовок.

Пример:

Сервис должен быть интегрирован с СК.

Что значит СК? Система Криптографии? Сервер Коммуникаций? Система Качества? Служба Консультирования? А что, если в компании есть все эти сервисы?

Однозначное требование:

Сервис должен быть интегрирован с Системой Контроля (СК), предоставляющей разграничение управления доступом.

5. ВЫПОЛНИМОСТЬ (или ОСУЩЕСТВИМОСТЬ).

Выполнимость требований в проекте определяется их возможностью быть реализованными с учетом технических, бюджетных и временных ограничений. Для оценки выполнимости можно использовать инкрементальную разработку и прототипы. Если требование невозможно выполнить, важно учитывать его влияние на проект и принимать решения о его исключении.

Примеры:

- 1. Приложение должно распознавать и выполнять мысленные команды пользователей.*
- 2. Приложение должно распознавать эмоции, по тексту и автоматически подбирать цветовой фон сообщения, соответствующий настроению пользователя.*

6. ПРОВЕРЯЕМОСТЬ (или ТЕСТИРУЕМОСТЬ).

Тестируемость требований означает их способность быть проверенными через объективные тест-кейсы, которые ясно показывают правильность реализации. Неполные, несогласованные или двусмысленные требования не поддаются проверке.

Пример 1: *Скидка Премиум-пользователя в размере 10% применяется по возможности к заказам пользователя, имеющего статус Премиум.*

Неясно, что означает "по возможности" и как проверить наступление или отсутствие этой возможности.

Проверяемое требование: Скидка Премиум-пользователя в размере 10% автоматически применяется ко всем заказам Премиум-пользователей.

Пример 2: *Система должна обладать высокой доступностью.* (Как это проверить ?)

Проверяемое требование: SLA * 99,99%

* SLA (Service Level Agreement) или соглашение об уровне предоставления услуги, выражаемый в процентах, описывает время безотказной работы и простоя системы, что относится к ее доступности. Например, SLA 99,99 означает, что система будет доступна 99,99% времени. Так, время безотказной работы составит в сутки 23 часа 59 минут 51 секунду, а допустимое время простоя – 9 секунд. В неделю время безотказной работы составит 6 дней 23 часа 59 минут, а допустимое время простоя – 1 минуту.

7. ПРИОРИТИЗИРОВАННОСТЬ (или УПОРЯДОЧЕННОСТЬ).

Требования должны быть упорядочены по важности, стабильности и срочности:

- **важность** определяет, насколько успех проекта зависит от выполнения требования;
- **стабильность** показывает, насколько вероятность изменений в требовании мала;
- **срочность** влияет на распределение усилий команды по времени.

Неверно расставленные приоритеты могут привести к неэффективному распределению ресурсов, выполнению ненужной работы и нарушению сроков.

Пример

1. Пользователь должен иметь возможность переключаться между светлой и темной темой интерфейса.

2. Система платежей должна быть интегрирована с новым партнером. API партнера может меняться по мере разработки.

3. Пользователь должен иметь возможность просмотреть информацию о своих заказах

Нарушена упорядоченность по важности и срочности, так как последнее требование намного полезнее для пользователя, чем наличие светлой и темной темы. Нарушена упорядоченность по стабильности, так как интеграция с системой, чья API может изменяться, создает высокую степень неопределенности и риск ненадежной работы функции, что может потребовать значительных доработок и тестирований. Реализацию этого требования можно отложить до релиза стабильной версии API партнером.

8. Атомарность (или "единичность")

Каждое требование должно быть самодостаточным и описывать только одну ситуацию. Если требование можно разбить на несколько независимых, оно перестаёт быть атомарным.

Проблемы с атомарностью возникают, когда в одном требовании описываются разные элементы интерфейса или разные состояния/эффекты от действий пользователя, что затрудняет его понимание и приводит к путанице.

Пример:

На странице отчёта должно отображаться имя пользователя, а также должна быть возможность скачивания PDF-версии отчёта.

Это требование описывает два разных элемента интерфейса и две разные функции программы: отображение имени пользователя и возможность скачать отчет.

А вот пример атомарных требований:

- На странице отчёта должно отображаться имя пользователя.*
- На странице отчёта должна быть кнопка для скачивания PDF-версии отчёта.*

9. Необходимость (или "обязательность")

Каждое требование должно приносить реальную пользу бизнесу, выделять продукт на рынке или обеспечивать соблюдение стандартов и правил. Если какое-то требование устарело, было замещено другим или просто не обязательно для реализации, его необходимо удалить из набора требований.

Пример 1:

Первоначальное требование: *"Пользователи должны иметь возможность вернуться на предыдущую страницу с помощью кнопки «Назад»."*

Новое требование: *"Пользователи должны иметь возможность вернуться на предыдущую страницу с помощью жеста свайпа."*

Кнопка «Назад» становится ненужной после внедрения жеста. Так как жест выполняет ту же функцию, но удобнее и соответствует современным стандартам пользовательского интерфейса. После удаления кнопки "Назад" должно быть удалено и требование о ней. Иначе будет путаница.

Пример 2 : *"Приложение должно поддерживать браузер Internet Explorer 10."* Это требование стало устаревшим и должно быть удалено, так как Internet Explorer 10 больше не используется и не поддерживается разработчиками.

10. Прослеживаемость (или "трассируемость")

Требования должны быть оформлены в структурированном виде и, в идеале, иметь уникальные идентификаторы. В контексте тестирования прослеживаемые (или "трассируемые") требования — это те, которые удобно связать с тестами. Для этого используются матрицы трассировки, которые будут рассмотрены позже.

"Приложение должно обеспечивать высокую безопасность, поддерживать различные уровни доступа для пользователей и администраторов, иметь возможность интеграции с внешними системами, такими как системы управления проектами; также оно должно работать на мобильных устройствах и поддерживать уведомления. Пользовательский интерфейс должен быть интуитивно понятным, с возможностью настройки тем, а производительность должна быть достаточной для работы с большими объемами данных. Важно обеспечить стабильную работу системы при одновременном подключении большого количества пользователей."

У такого полотна требований отсутствует прослеживаемость, потому что:

- нет структуры, выделяющей функциональные и нефункциональные требования;
- отсутствует нумерация требований;
- несколько требований смешаны в одном абзаце, что затрудняет отслеживание и реализацию;
- нет четкого деления по категориям (безопасность, интерфейс, производительность и т.д.).

11. Модифицируемость

Модифицируемость требований подразумевает легкость их изменения. Если требования к продукту разбросаны по разным хранилищам или одно и то же требование встречается в нескольких местах, это значительно усложняет их изменение. Хранение требований в единой базе и использование уникальных идентификаторов помогают избежать избыточности и облегчить управление изменениями.

Пример:

В разделе "Авторизация" в Notion: "Все пользователи системы должны проходить двухфакторную аутентификацию для доступа к личным данным."

В разделе "Безопасность" в Confluence: "Все пользователи системы должны проходить двухфакторную аутентификацию для доступа к личным данным."

Плохо: Одно и то же требование описано как в разделе безопасности, так и в разделе авторизации, ещё и в двух разных хранилищах. При изменении требования в одном месте (например, добавлении новых условий аутентификации) возникает большой риск появления противоречий с другими частями требований. Чтобы внести изменения, придётся прочесывать все хранилища и перечитывать требования, а затем изменять их в нескольких местах. Это увеличивает трудозатраты и время на внесение изменений.

12. Понятность

Понятность определяется тем, насколько легко требования понимает целевая аудитория. Требования должны быть написаны с использованием терминологии, знакомой всем членам команды, которая их использует в работе. Это позволит избежать недоразумений и неправильной интерпретации. Если требования описаны неясно или с использованием специального жаргона, который не является общепринятым, это усложняет их понимание, выполнение и замедляет онбординг новых сотрудников.

Пример:

"Сайт должен быть способен к улучшению взаимодействия через динамическое управление параметрами."

Фразы "улучшение взаимодействия" и "динамическое управление параметрами" слишком абстрактны и не конкретизируют, какие аспекты взаимодействия нужно улучшить и какие параметры должны быть динамически управляемыми. Требование не несёт полезной информации.

Понятное требование:

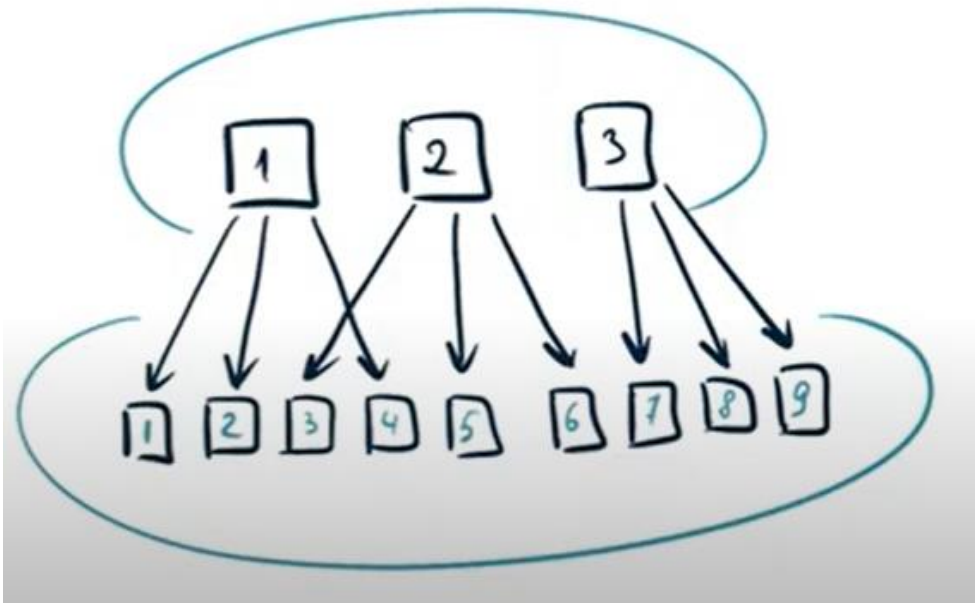
Сайт должен предоставлять возможность настройки пользовательского интерфейса в реальном времени: переключение светлой и тёмной темы, три варианта величины шрифта, переключение в режим высокой контрастности."

Трассировка требований (прослеживаемость)

Трассировка — это способ представления отношений между требованиями различного уровня в системе, помогающий определить источник любого требования.

Трассируемость – это связь требования с уровнем выше и уровнем ниже.

Бизнес требования Заказчика



Требования нижнего уровня
(спецификация, ТЗ)

Трассировка требований позволяет:

- подтвердить, что реализация удовлетворяет всем требованиям: все, что требовал заказчик, было реализовано;
- подтвердить, что приложение делает только то, что было заказано: не реализовано то, что заказчик никогда не просил;

Зачем нужна трассировка требований?

<https://www.youtube.com/watch?v=IU-2r89pgvA>

- Анализ важности новых требований

При появлении новых требований можно проанализировать удовлетворяет ли новое требование бизнес-цели и представляет ли оно ценность для нашего продукта

- Анализ изменений

При изменении какого-либо требования верхнего уровня по трассировке можно проанализировать какие требования нижнего уровня изменятся и какие изменения нужно в них внести, что бы обеспечить изменение требования верхнего уровня

- Управление приоритетом

Пример:

У проекта 3 бизнес-цели

У каждой бизнес-цели по 4 бизнес требования

У каждого бизнес-требования по 8 пользовательских историй

Пользовательские истории можно приоритизировать по приоритету связанных с ними бизнес-целей



Матрица трассируемости (RTM — Requirements Traceability Matrix)

Трассируемость требований осуществляется при помощи матриц трассируемости, которые показывают связь между элементами.

Матрицы трассируемости могут быть различными для разных уровней требований, например:

- Бизнес-требования → Функциональные требования
- Функциональные требования → Тест-кейсы

Пример матрицы трассируемости

ID Бизнес-требования	Бизнес-требование	ID Функционального требования	Функциональное требование
BR-001	Клиент должен иметь возможность быстро проверять баланс счёта без входа в полную версию приложения	FR-101	Виджет баланса на главном экране с биометрической авторизацией
		FR -102	Push-уведомление о изменении баланса при запросе
BR -002	Клиент должен иметь возможность переводить деньги между своими счетами в 1-2 касания	FR -201	Экран выбора счетов «Откуда» / «Куда»
		FR -202	Шаблоны частых переводов с иконками
		FR -203	Подтверждение перевода по FaceID/TouchID

Пример составления TRACEABILITY MATRIX (для части функционала Калькулятора) для элементов

Функциональные требования → Тест-кейсы

ЭТАП 1: Составляем нумерованный список требований к приложению (для примера возьмём несколько существующих функций калькулятора)

<u>ID требования</u>	<u>Формулировка требования</u>
Rq1	Ввод цифр и действий. Можно производить вычисления, нажимая на кнопки калькулятора или вводя символы с клавиатуры. Кроме того, доступен ввод цифр и действий с цифровой клавиатуры, когда нажата клавиша NUM LOCK. Также можно вставлять математические выражения из буфера обмена и получать результат (например, набрать в Блокноте «5*3=», скопировать и вставить в Калькулятор, на «экране» которого появится ответ «15»).
Rq2	Очистка экрана / Удаление символов. Полная очистка экрана осуществляется при нажатии на кнопку «C» калькулятора или клавишу Delete на клавиатуре. Последний введенный числовой символ можно удалить, используя клавишу Backspace на клавиатуре.
Rq3	Калькулятор можно использовать для выполнения операций сложения.

ЭТАП 2: Формируем список тестов – это может быть список названий детально расписанных пошаговых тест-кейсов или же чек-лист проверок

<u>ID теста</u>	<u>Проверка</u>
Th1	Ввод значений и операций при нажатии на кнопки калькулятора
Th2	Ввод цифр и действий с цифровой клавиатуры, когда нажата клавиша NUM LOCK
Th3	Ввод цифровых значений с клавиатуры
Th4	Вставка математических выражений из буфера обмена (на одно, два, максимально возможное количество действий)
Th5	Очистка экрана нажатием на кнопку «C» калькулятора
Th6	Очистка экрана нажатием на клавишу Delete калькулятора
Th7	Удаление последнего введенного символа нажатием на клавишу Backspace на клавиатуре
Th8	Сложение однозначных, двузначных, многозначных чисел
Th9	Сложение десятичных чисел
Th10	Сложение отрицательных чисел

ЭТАП 3: Составляем TRACEABILITY MATRIX, которая может быть представлена разными способами

TRACEABILITY MATRIX	
ID требования	ID теста
Rq1	Th1
	Th2
	Th3
	Th4
Rq2	Th5
	Th6
	Th7
Rq3	Th8
	Th9
	Th10

TRACEABILITY MATRIX												
	ID теста	Th1	Th2	Th3	Th4	Th5	Th6	Th7	Th8	Th9	Th10	Кол-во тестов, которые покрывают требование
ID требования												
Rq1		X	X	X	X							4
Rq2						X	X	X				3
Rq3									X	X	X	3

Таким образом матрица трассируемости:

- позволяет **контролировать реализацию требований**, отслеживать, что все требования разработаны и протестированы и ничего не пропущено.
- помогает **отслеживать какие именно требования еще не покрыты тест-кейсами**.
- используется **для контроля измененных требований**.
- могут использоваться и заказчиком, что позволяет сделать процесс разработки и тестирования более прозрачным.